National University of Sciences and Technology

School of Mechanical and Manufacturing Engineering

# RAI-833 Deep Learning

## Tutorial 8: Object Detection using Faster R-CNN

*Pre-Trained Model Inference and Evaluation Metrics*

*Submitted to:*

**Dr. Shahbaz Khan**

*Submitted by:*

**Hamza Suhaib Qarni**

**538866**

**SMME**

April 12, 2026

# 1 Objectives

The primary objective of this tutorial is to deploy a pre-trained, state-of-the-art object detection model to perform inference on novel images without requiring custom training. Key learning outcomes include:

- Loading and configuring PyTorch's pre-trained **Faster R-CNN** model.
- Transforming raw image data into multi-dimensional tensors for model ingestion.
- Post-processing raw model predictions using confidence thresholding.
- Understanding the mathematical foundations of model evaluation, specifically **Intersection over Union (IoU)** and **Mean Average Precision (mAP)**.

# 2 Methodology

Instead of initializing random weights and training from scratch, this tutorial leverages **Transfer Learning** via a pre-trained model.

## 2.1 Model Configuration

The model architecture is a **Faster R-CNN** utilizing a **ResNet-50** backbone combined with a **Feature Pyramid Network (FPN)**. The model was initialized with `DEFAULT` weights trained on the extensive COCO (Common Objects in Context) dataset, allowing it to immediately recognize 91 distinct object categories.

Table 1: Pre-Trained Model Specifications

| **Specification** | **Value** |
|---|---|
| Architecture | Faster R-CNN |
| Backbone | ResNet-50 + FPN |
| Weights | `DEFAULT` (COCO V1) |
| Total Classes Supported | 91 (e.g., Person, Car, Bicycle) |
| Inference Mode | `model.eval()` + `torch.no_grad()` |

## 2.2 Image Pre-Processing

To perform inference, the raw pixel data of a custom street scene image (`street scene.jpeg`) was converted into a PyTorch Tensor. A dummy batch dimension was added using `unsqueeze(0)` to satisfy the model's input shape requirements, resulting in a final tensor shape of $[1, 3, 480, 852]$ (Batch, Channels, Height, Width).

# 3 Evaluation Metrics Framework

To assess the quality of object detection algorithms objectively, two primary mathematical metrics are utilized:

## 3.1 Intersection over Union (IoU)

IoU is the mathematical ratio used to determine the **spatial accuracy** of a bounding box prediction compared to the ground truth. It is calculated by dividing the area of overlap by the total area encompassed by both boxes (where $A$ is the predicted box and $B$ is the ground truth box):

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}} = \frac{|A \cap B|}{|A \cup B|} \tag{1}$$

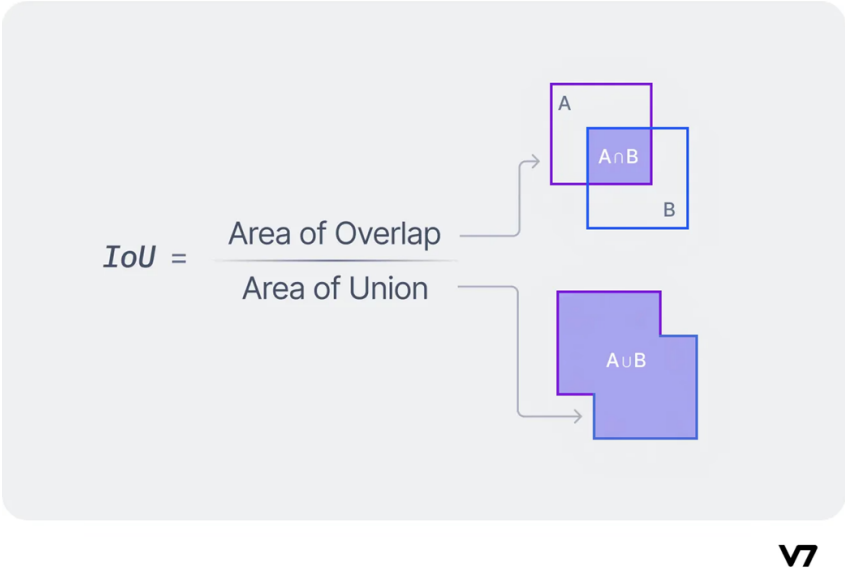


Figure 1: IoU Intersection over Union Demonstration.

To demonstrate this, a manual Python function was implemented to calculate the exact spatial overlap:

```python
def calculate_iou(boxA, boxB):
# Determine intersection coordinates
xA, yA = max(boxA[0], boxB[0]), max(boxA[1], boxB[1])
xB, yB = min(boxA[2], boxB[2]), min(boxA[3], boxB[3])

# Compute intersection area
interArea = max(0, xB - xA) * max(0, yB - yA)

# Compute union area
boxAArea = (boxA[2] - boxA[0]) * (boxA[3] - boxA[1])
boxBArea = (boxB[2] - boxB[0]) * (boxB[3] - boxB[1])
unionArea = float(boxAArea + boxBArea - interArea)

return interArea / unionArea
```

Listing 1: Manual PyTorch implementation of the IoU mathematical formula.

*Testing this function on dummy coordinate data during the tutorial yielded an IoU score of* ***0.6806****.*

### 3.2 Mean Average Precision (mAP)

While IoU evaluates a single bounding box, mAP evaluates the detection performance of the entire model. To calculate this, we must first define **Precision** and **Recall**:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \tag{2}$$

where $TP$ represents True Positives, $FP$ represents False Positives, and $FN$ represents False Negatives.

**Average Precision (AP)** is calculated by determining the area under the Precision-Recall curve for a specific class:

$$AP = \int_0^1 P(R)\, dR \tag{3}$$

Testing the AP calculation logic on a dummy set of predictions yielded an Average Precision of **0.8500**. Finally, **Mean Average Precision (mAP)** is the calculated mean of the AP scores across all $N$ classes (such as the 91 classes in the COCO dataset):

$$mAP = \frac{1}{N} \sum_{i=1}^{N} AP_i \tag{4}$$

## 4 Results and Visualization

During the forward pass, the pre-trained model processed the street scene tensor and initially proposed **100 potential objects**. Because models assign probabilities to every anchor box, a strict confidence threshold of **70%** ($> 0.70$) was applied to eliminate false positives and background noise.

Filtering the raw tensor outputs reduced the predictions down to **10 highly confident detections**.

Table 2: Sample Detections from Inference ($> 70\%$ Confidence)

| **Detection Instance** | **Predicted Class** | **Confidence Score** |
|---|---|---|
| Instance 1 | Car | $> 99\%$ |
| Instance 2 | Car | $> 99\%$ |
| Instance 3 | Person | $> 98\%$ |
| Instance 4 | Person | $> 97\%$ |
| Instance 5 | Bus | $> 96\%$ |

### 4.1 Visual Output

Using the `matplotlib` and `patches` libraries, the bounding box coordinates $[x_{min}, y_{min}, x_{max}, y_{max}]$ were mapped back to the original image space. The bounding boxes, class labels, and confidence percentages were dynamically drawn, proving the model's zero-shot capability to accurately isolate objects in a dense environment.

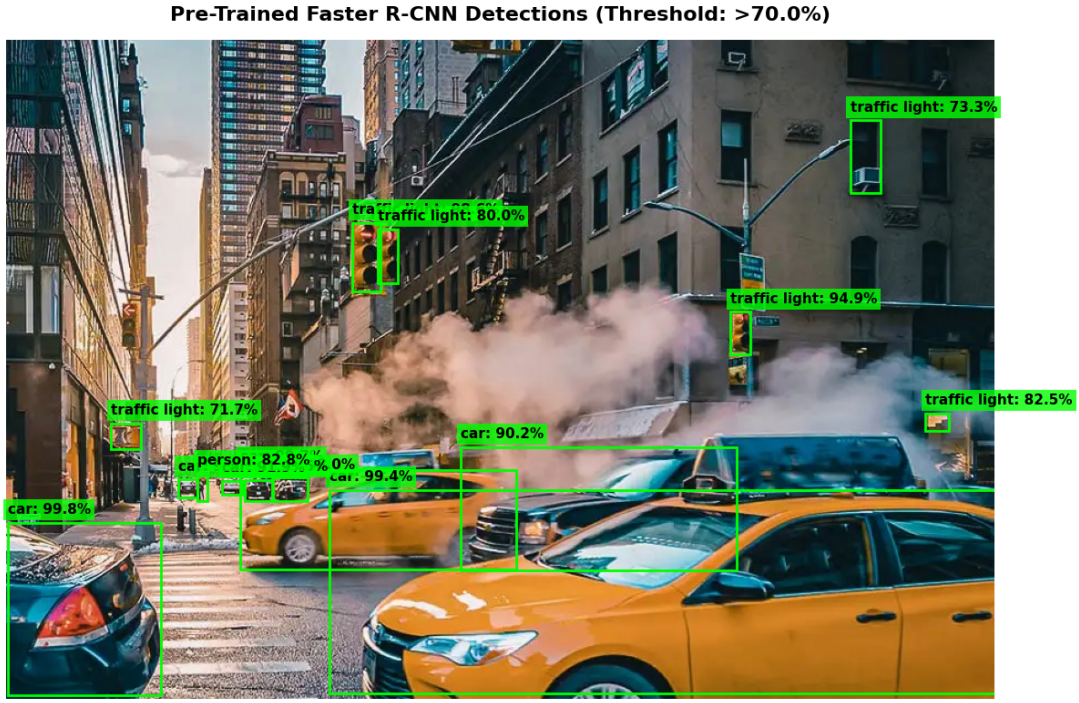


Figure 2: Inference results on a custom street scene using the pre-trained COCO weights, showcasing precise localization of vehicles and pedestrians.

# 5 Conclusion

This tutorial effectively demonstrated the immense utility of **Transfer Learning** in Computer Vision. By bypassing the computationally expensive training phase, the pre-trained Faster R-CNN model was immediately deployed for high-accuracy inference, successfully detecting 10 distinct targets in an unseen street scene. Furthermore, by programmatically defining IoU and mAP calculations, a comprehensive understanding of how object detection architectures are validated was firmly established.